

Distributed Systems

Section 5. RPC and Distributed Objects

Pedro García López

pgarcia@etse.urv.es/

Copyright

- © University Rovira i Virgili
- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; A copy of the license is available at:
 - <http://www.fsf.org/licenses/fdl.html>

TOC

1. Interfaces
2. Communication model
3. Naming
4. Use Cases

Introduction

Applications

RMI, RPC and events

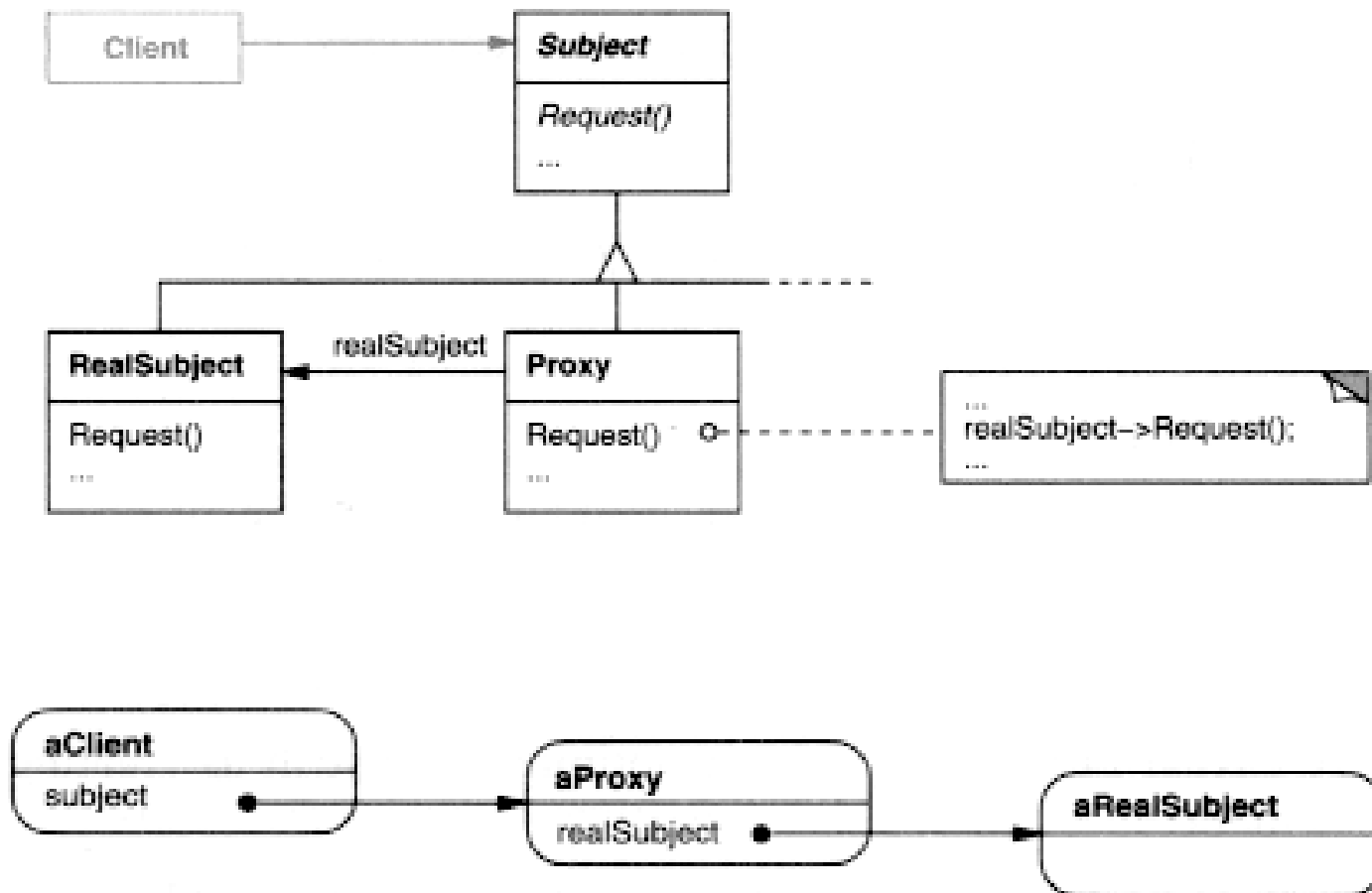
Request Reply protocol

Marshalling and data encoding

Operating System

Middleware

Design Pattern: Proxy



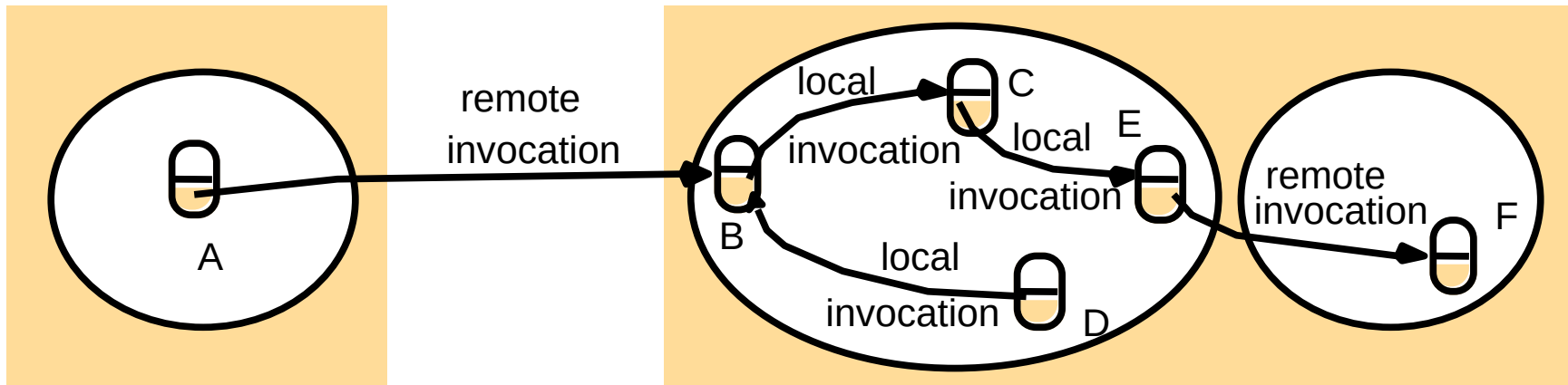
1. Interfaces

- Interface Definition Language
 - Sun RPC XDR
 - CORBA IDL
 - DCE IDL
 - Microsoft MIDL
 - Java Interfaces
- Remote Service Definition
 - Methods or procedures
 - Types of parameter passing (value or reference)

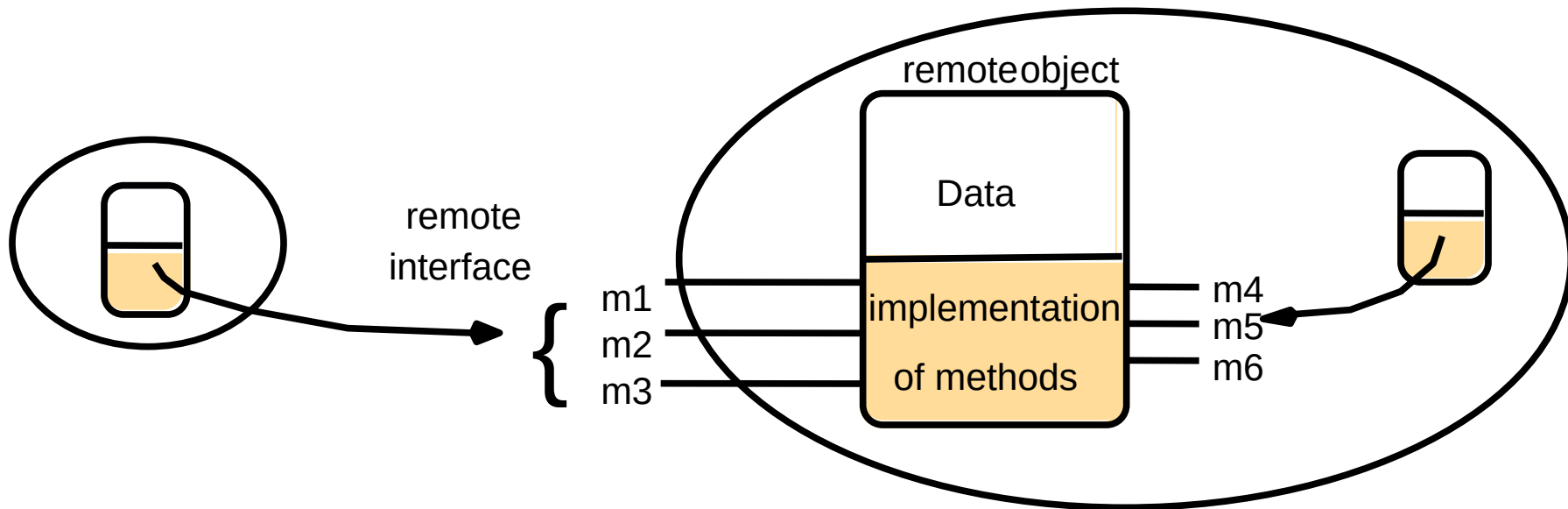
Interfaces: CORBA IDL

```
// En el archivo Persona.idl  
struct Persona {  
    string nombre;  
    string lugar;  
    long año;  
};  
interface ListaPersonas {  
    readonly attribute string nombrelista;  
    void añadePersona(in Persona p) ;  
    void damePersona(in string nombre, out Persona p);  
    long número();  
};
```

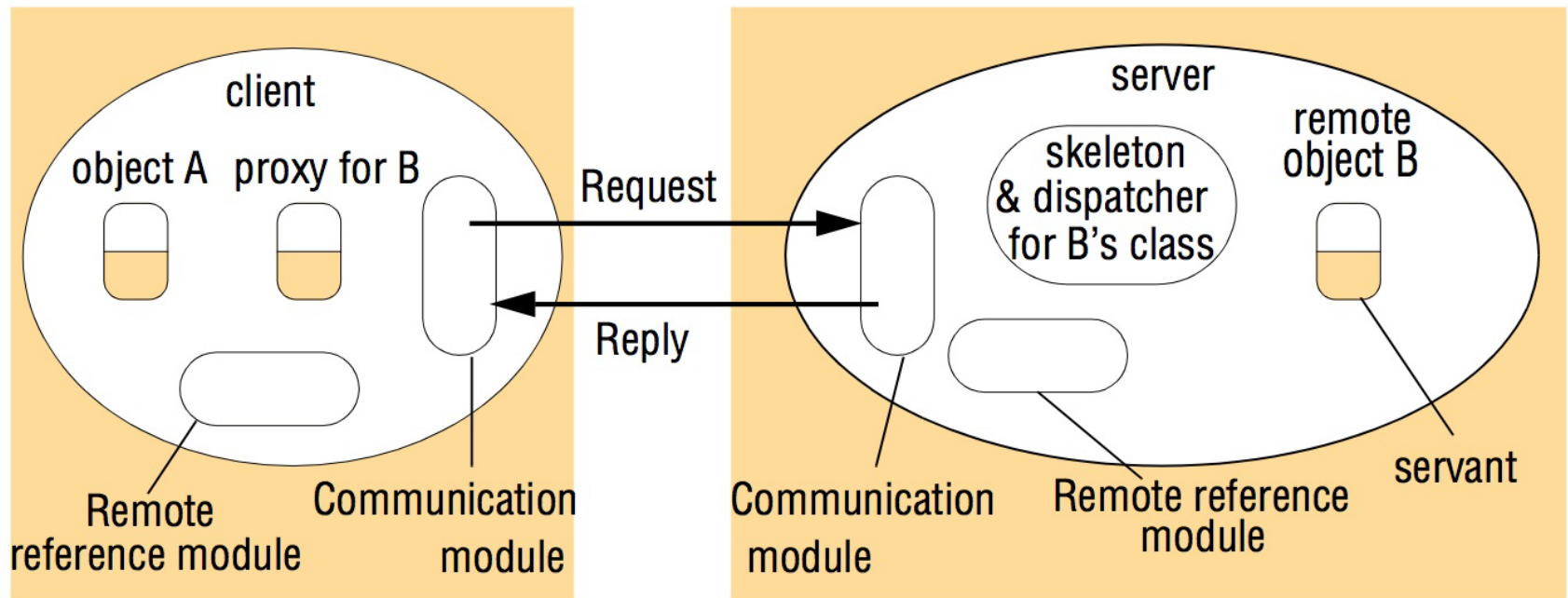
Local and remote invocation



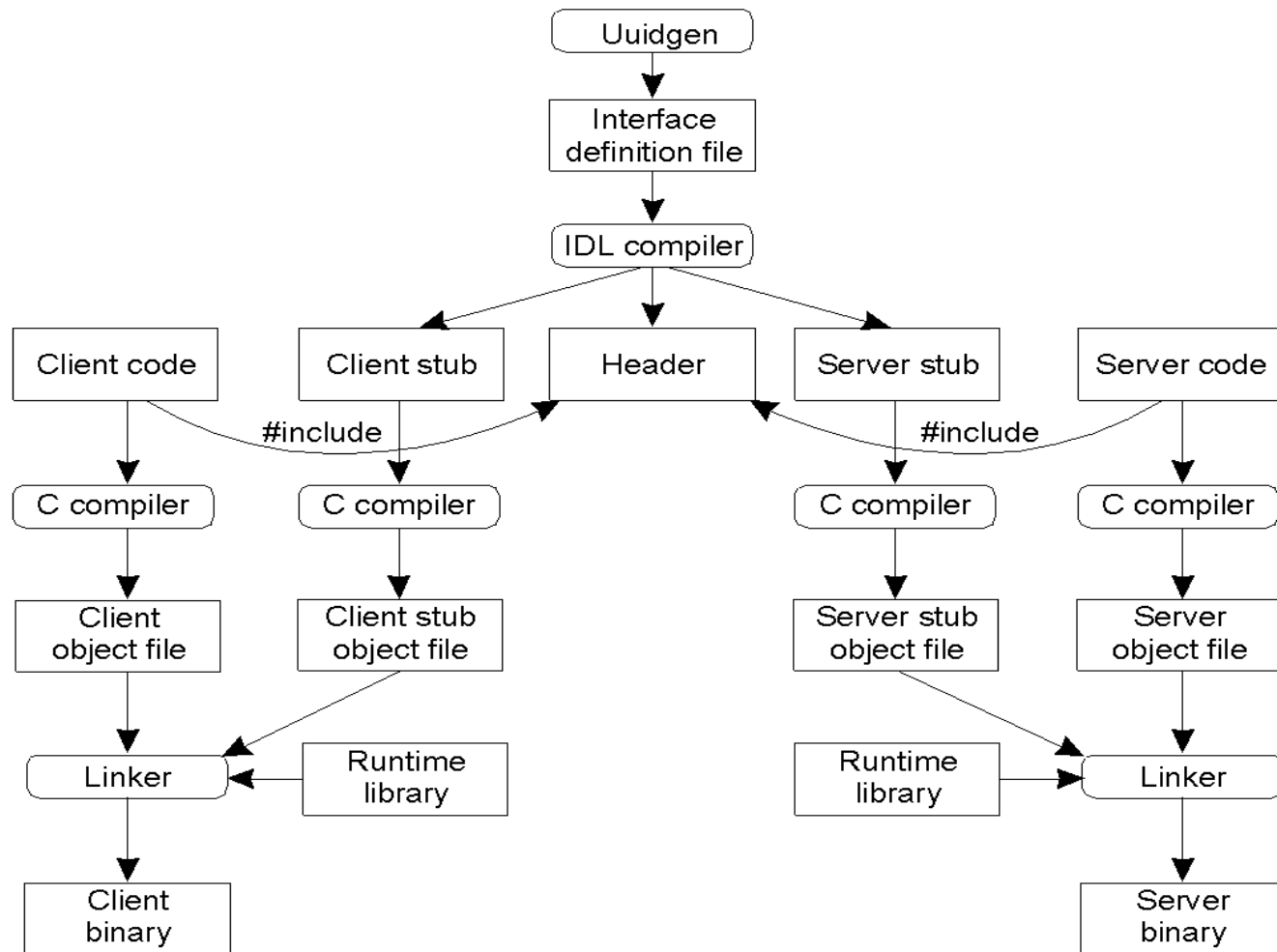
Remote interfaces



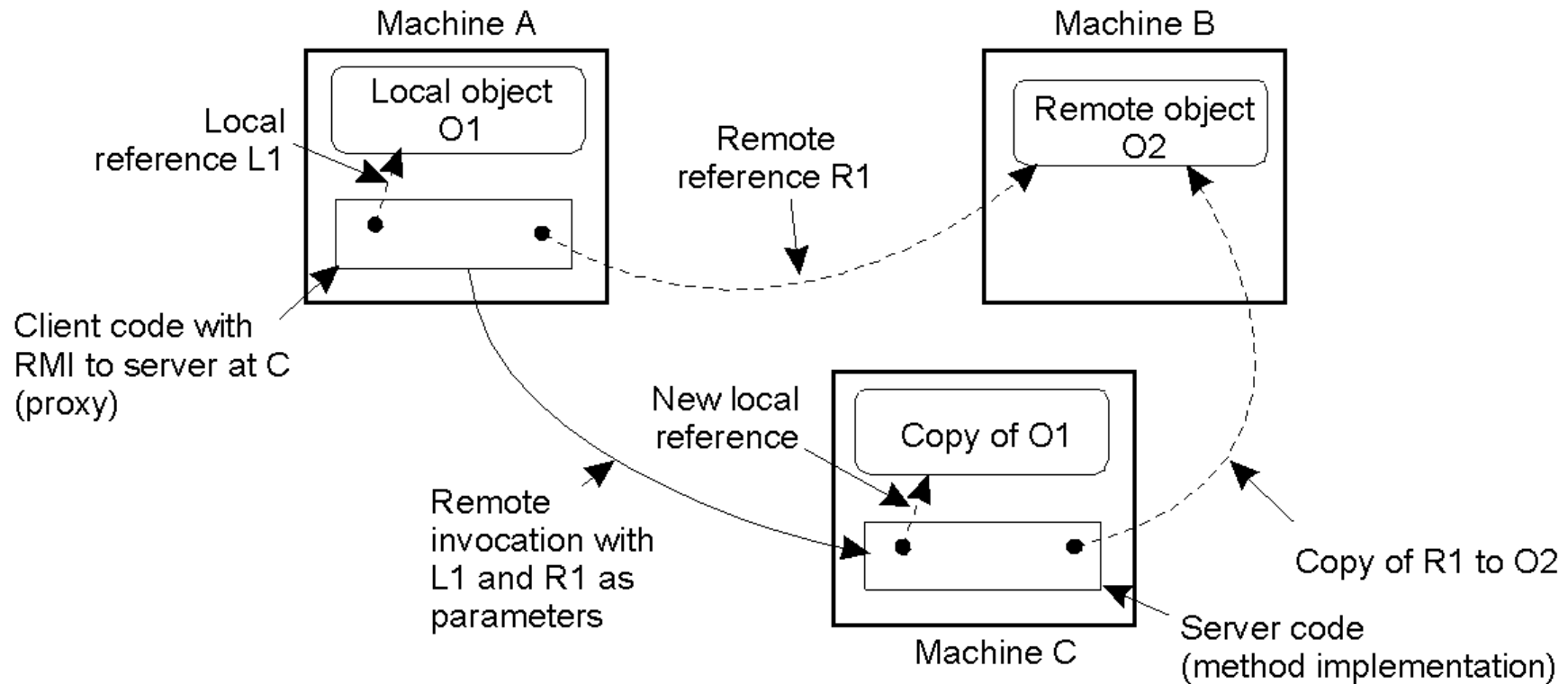
The proxy



DCE RPC IDLs

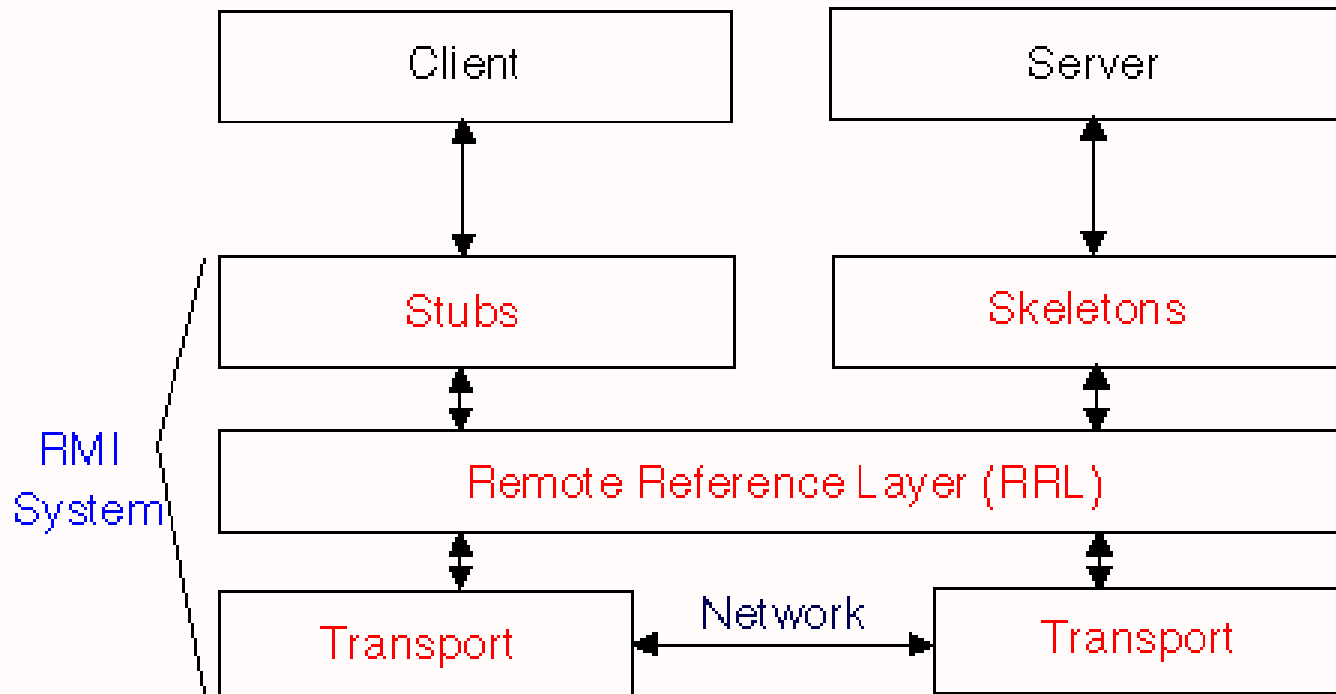


Parameter passing

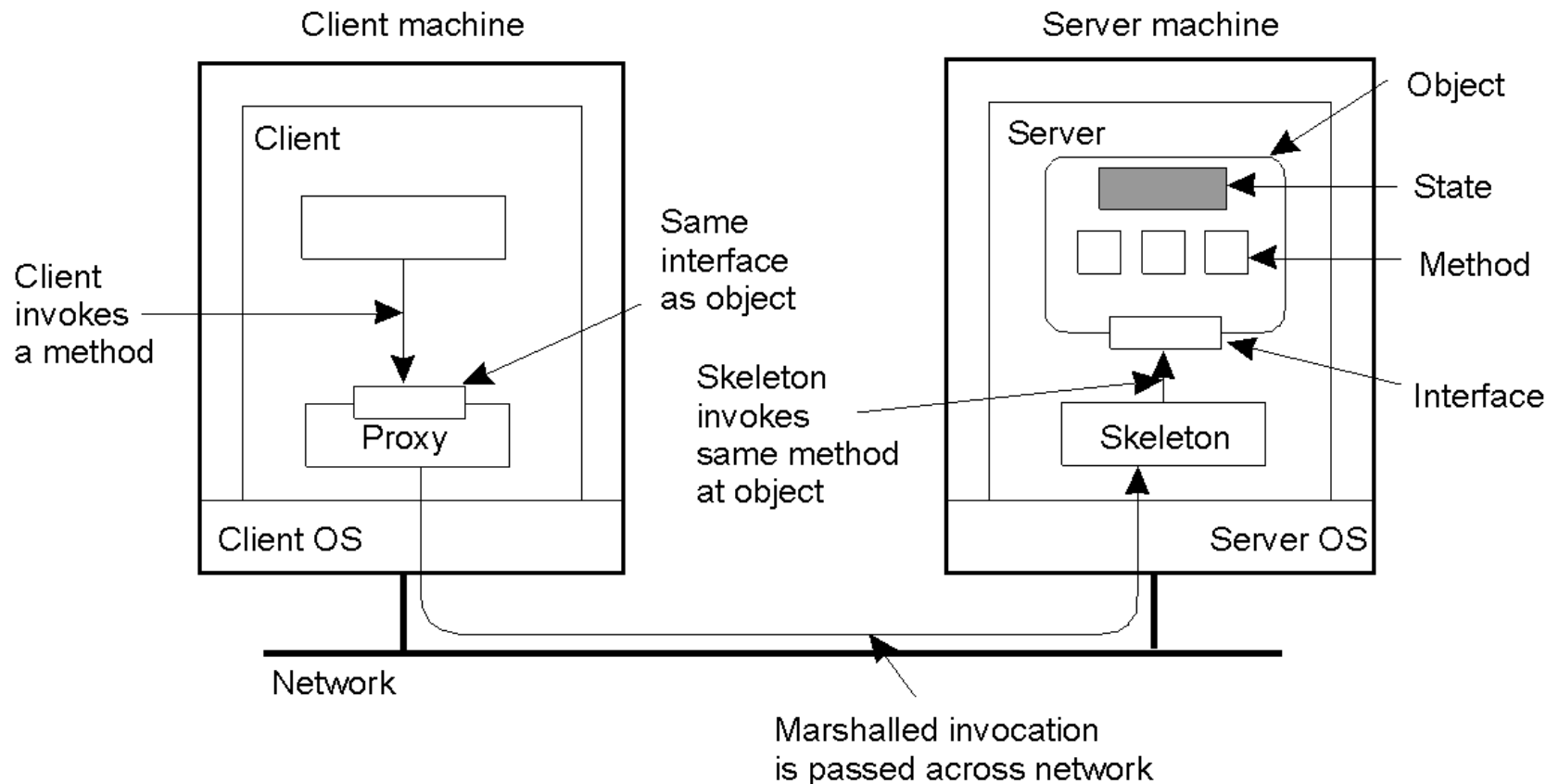


2. Communication model

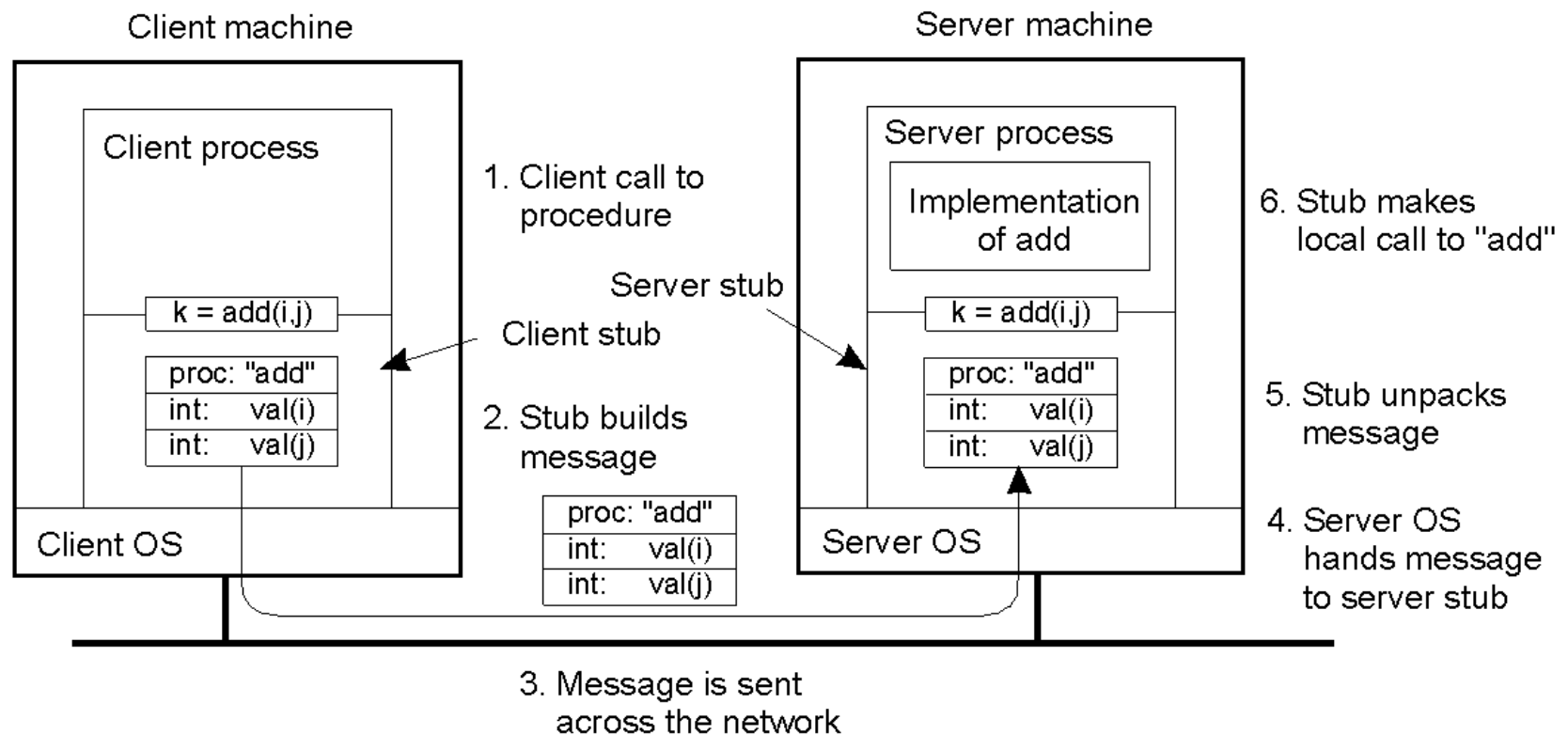
Application



Distributed Objects



Communication model



Marshalling

- a) Procedure
- b) Message

```
foobar( char x; float y; int z[5] )  
{  
    ....  
}
```

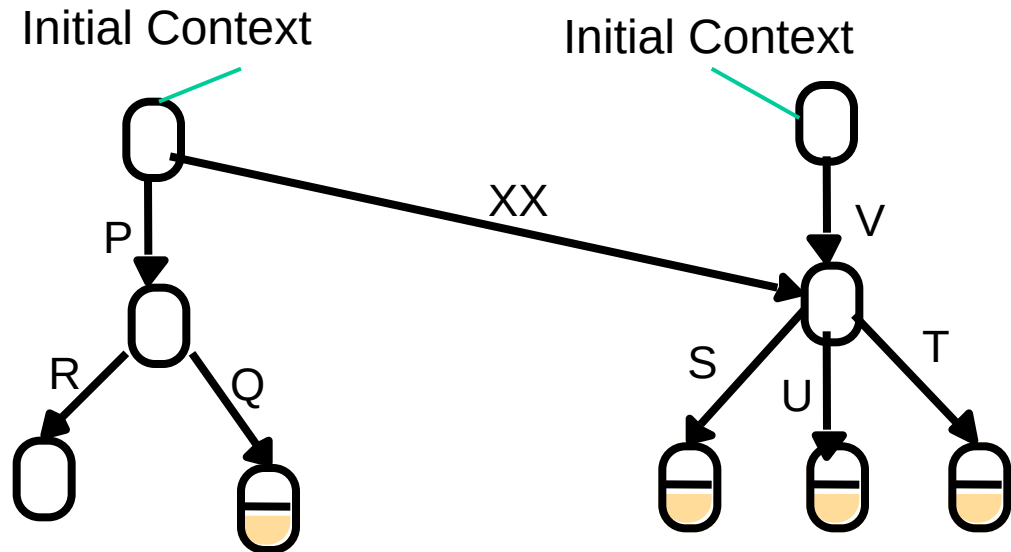
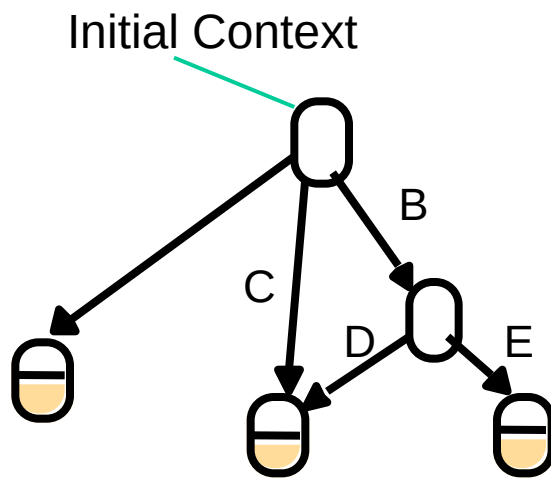
(a)

foobar's local variables	
	x
y	
5	
z[0]	
z[1]	
z[2]	
z[3]	
z[4]	

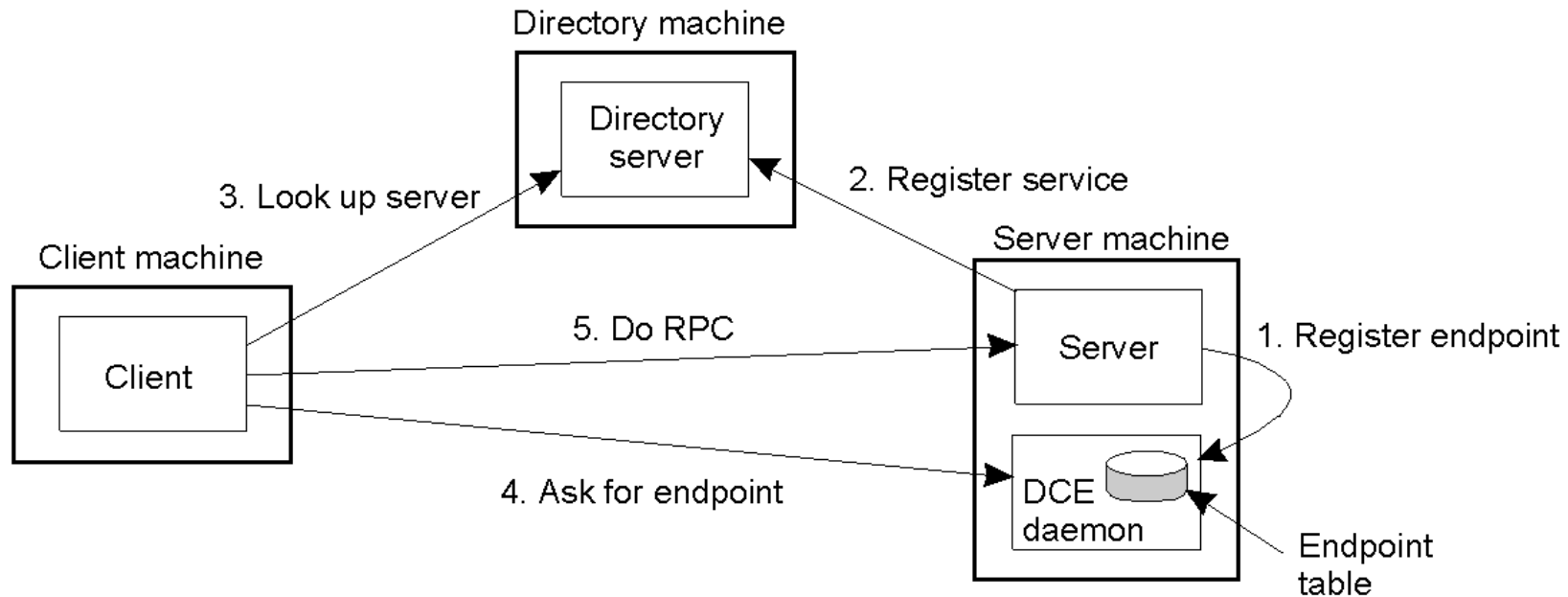
(b)

Naming

- CORBA Naming Service
- Java RMIregistry
- LDAP
- Active Directory



Register and lookup



4. Use Cases

- OMG CORBA
- Microsoft COM+
- Java RMI
- XMLRPC

CORBA

- OMG standard (Object Management Group)
- Remote method invocation
- Platform and language independent
- Requires an IDL (Interface Description Language)

Architecture

- ORB (Object Request Broker)
 - Remote method invocation
 - Hides the location of objects
 - Manages request and replies
- Object Services
 - Basic object services
 - Names, Events, Transactions, etc...

Remote Invocation

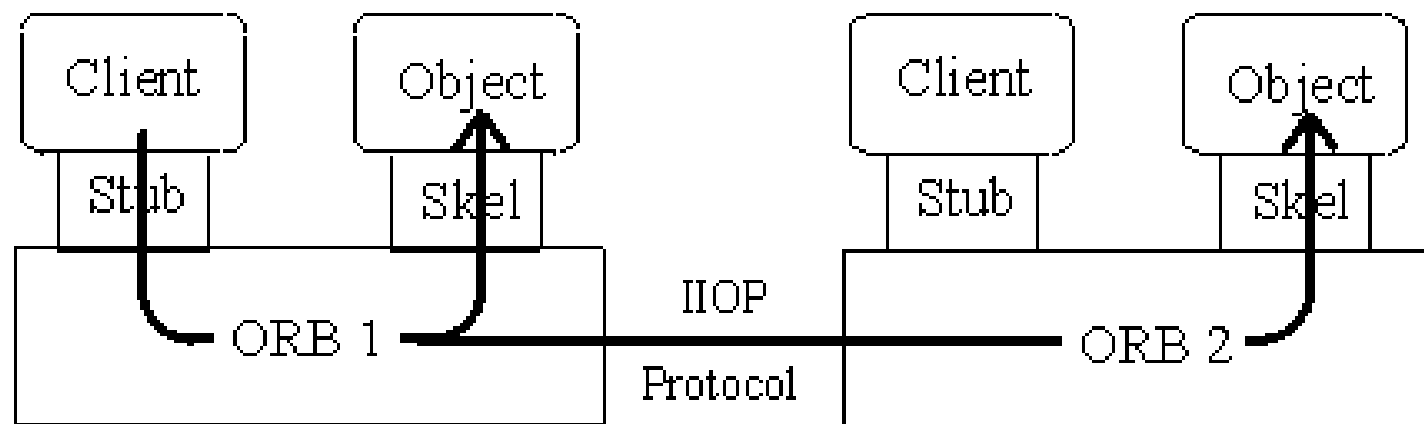
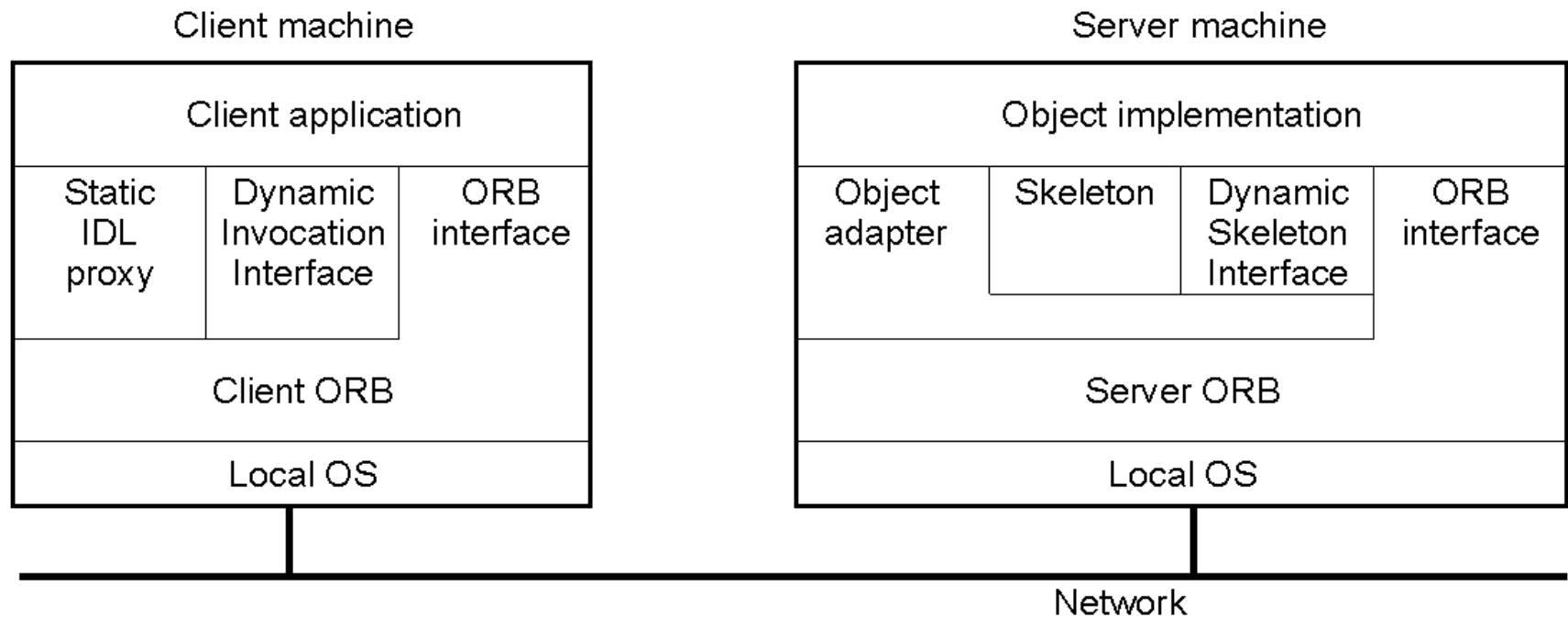


Figure 2: Interoperability uses ORB-to-ORB communication

Copyright © 2000 Object Management Group

Architecture



ORB

- It can be different in every machine
- It must comply with the CORBA protocol
- Hides implementation details
- Connects clients and servers

HelloWorld

- File .idl:

```
//IDL
interface Hello {
    void sayHello();
    string giveMeHello();
};
```

- Generate stubs & skels
 - `jidl --package HelloPkg --impl Hello.idl`

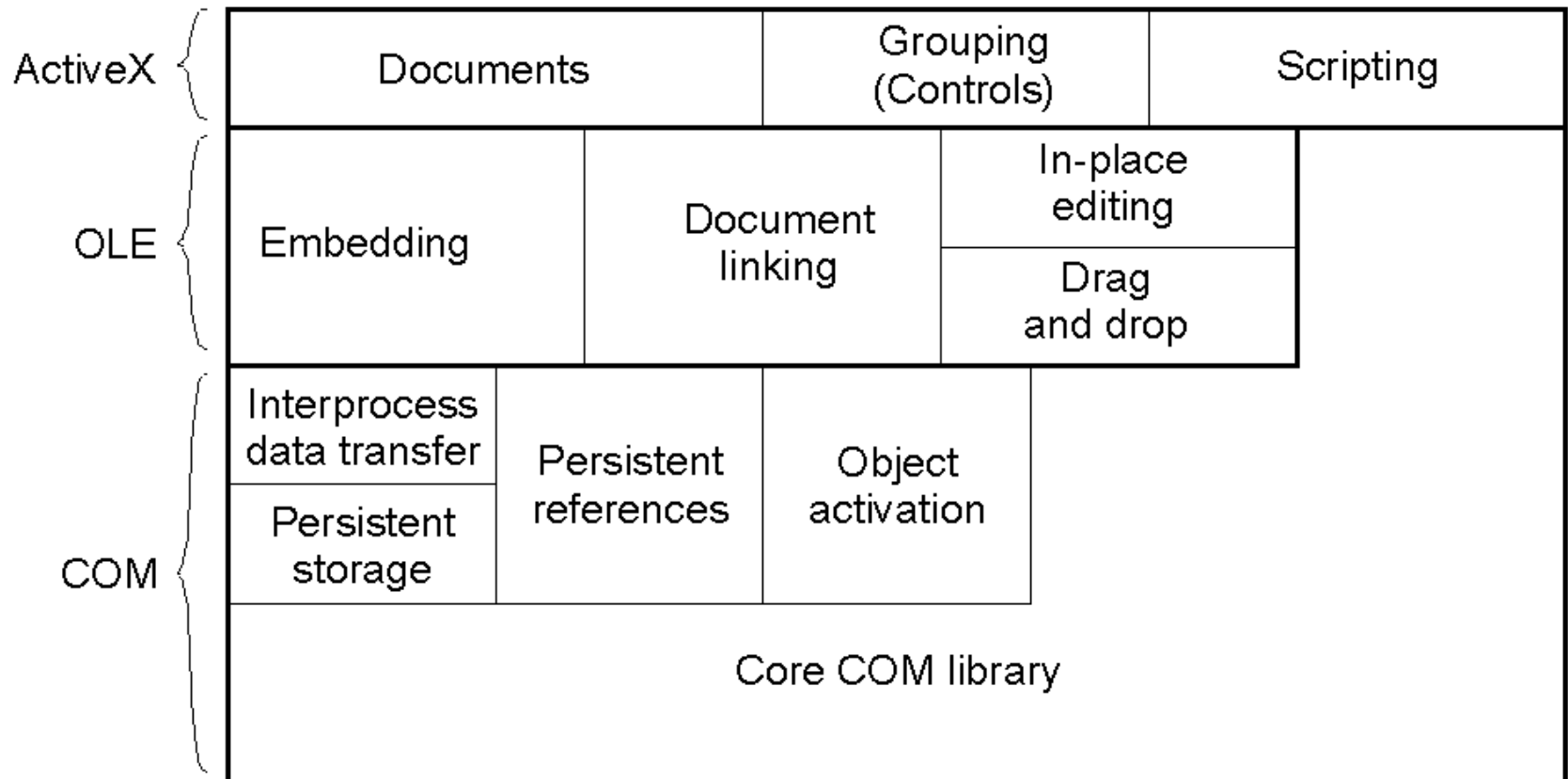
HelloWorld

- Files:
 - Hello.java
 - HelloHelper.java,
 - HelloHolder.java
 - HelloOperations.java
 - HelloPOA.java
 - _HelloStub.java
 - Hello_impl.java (opcional)

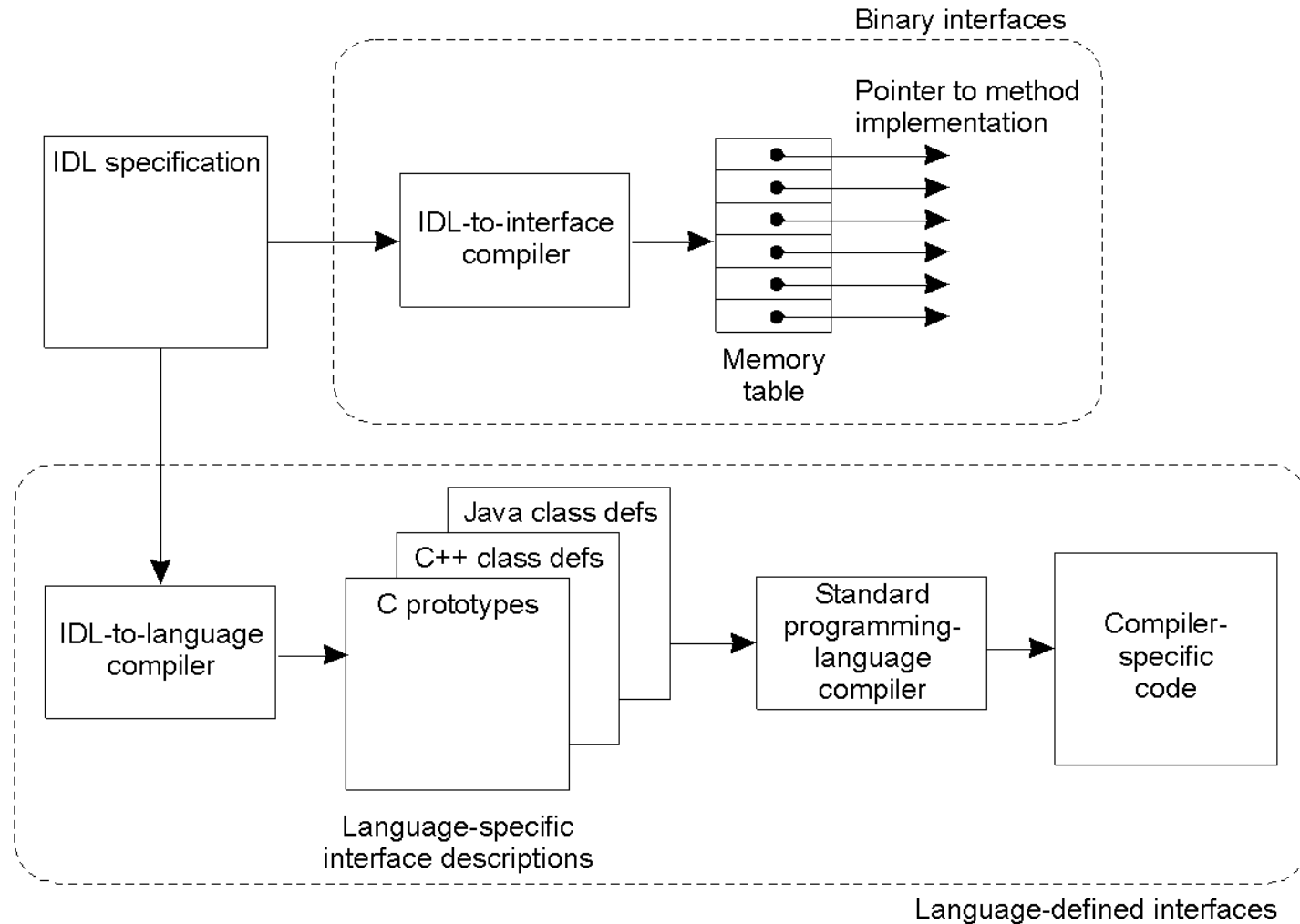
CORBA Services

- Naming
- Events
- “Life Cycle”
- Persistence
- Transactions
- Concurrency
- Relationships
- Externalization
- Query
- Licensing
- Properties
- Time
- Security
- Collections

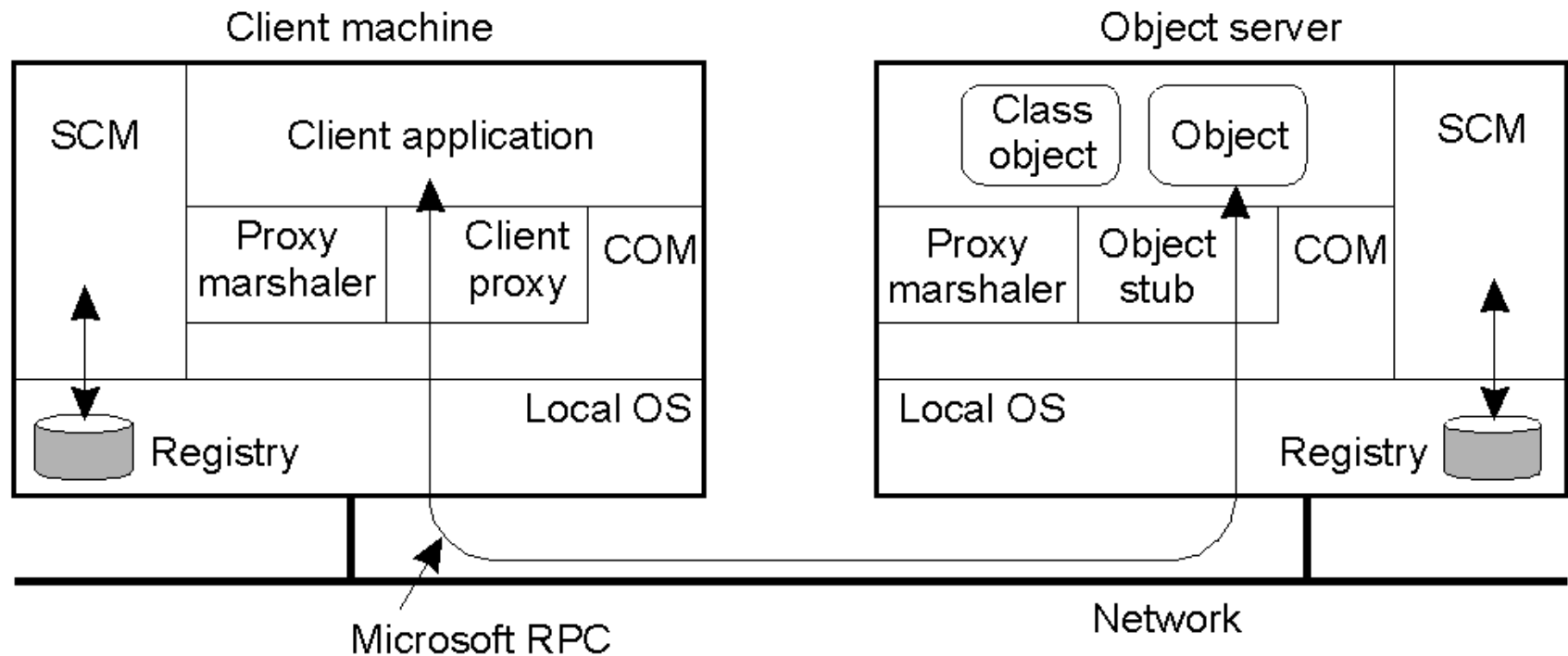
COM & COM+



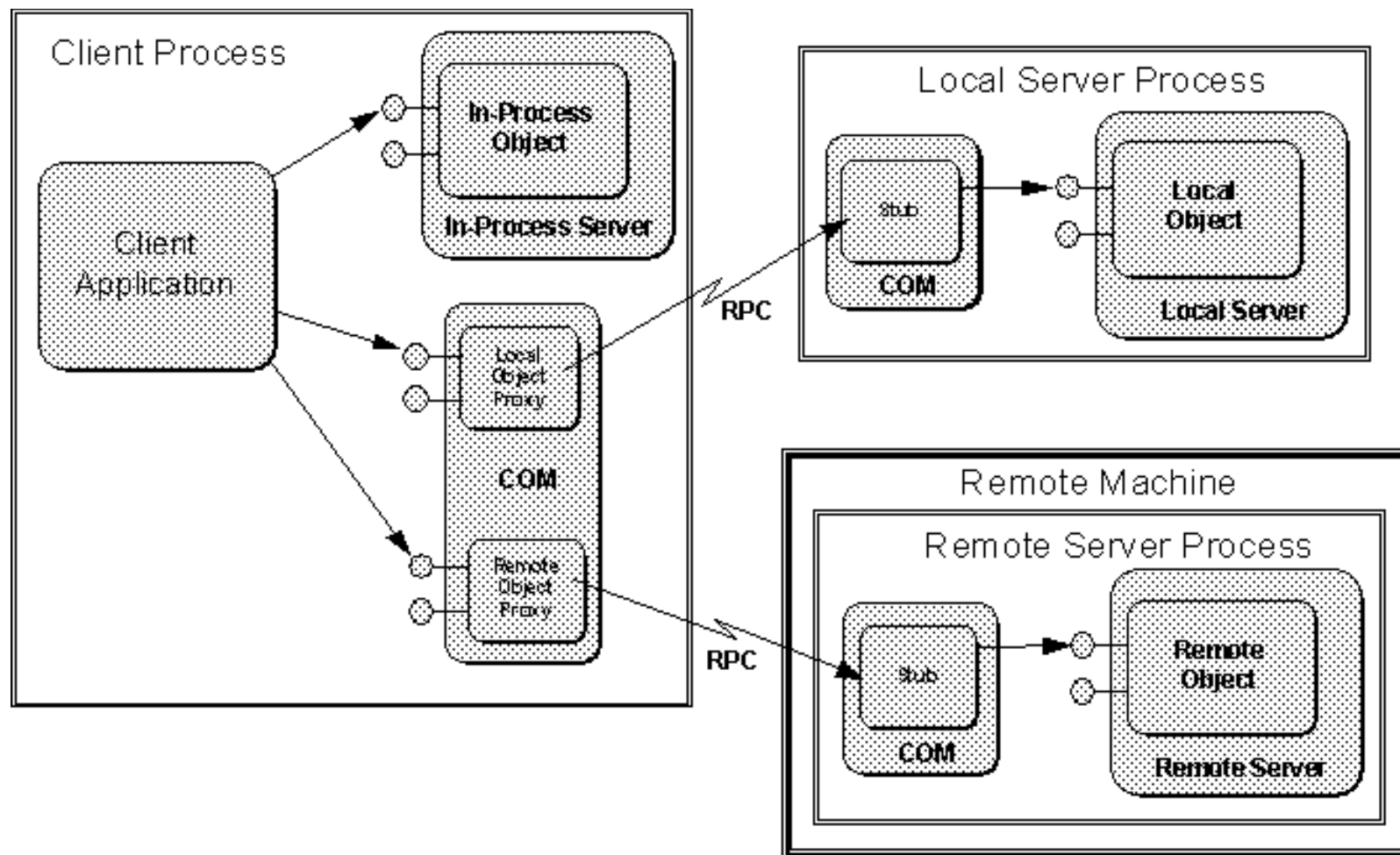
Architecture



Type Library & Registry



COM+

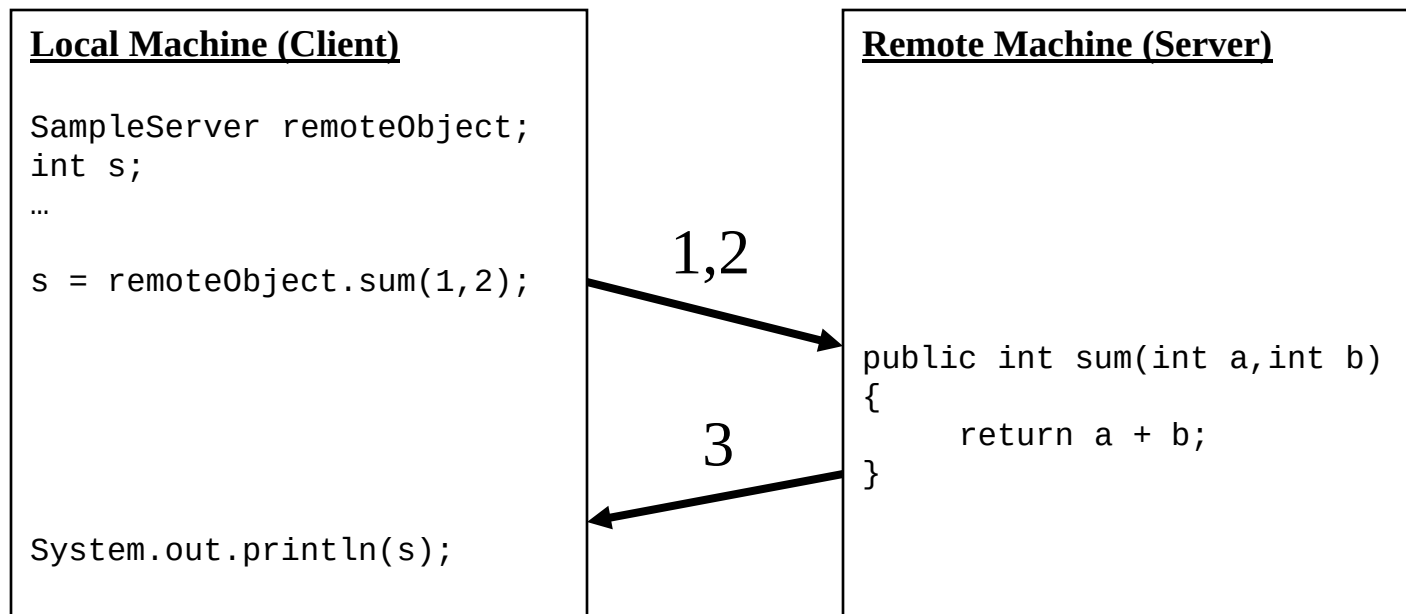


Remote Method Invocation RMI

Corba versión Java

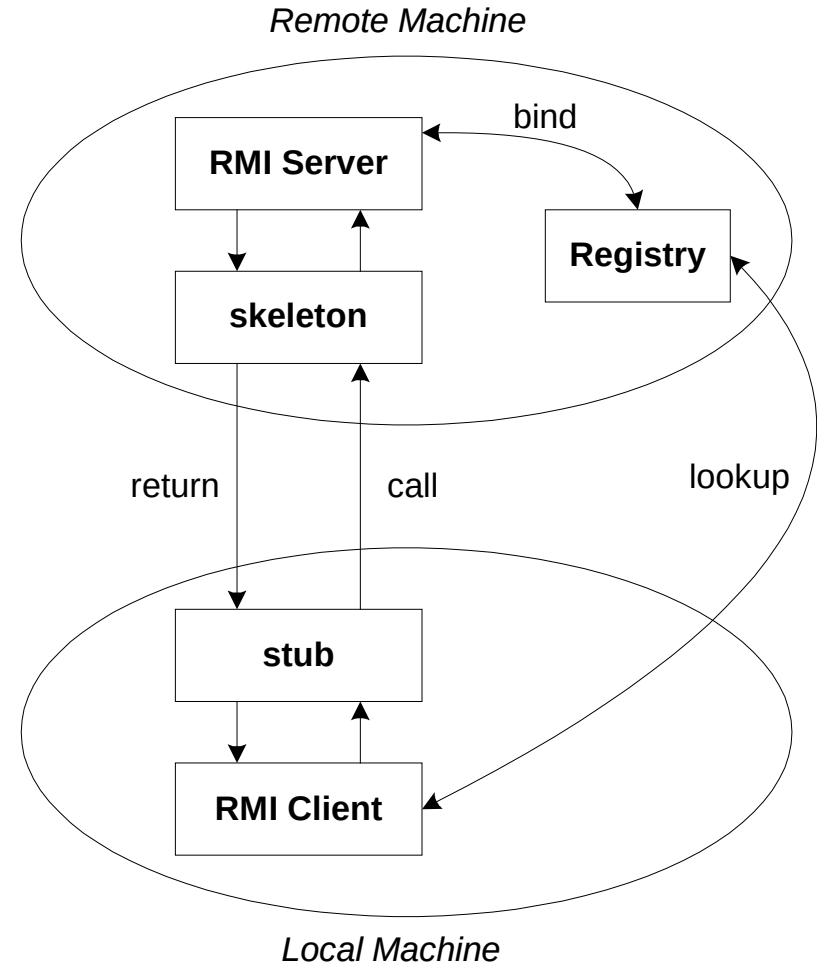
Java Remote Object Invocation (RMI)

- Overview of RMI
- Java RMI allowed programmer to execute remote function class using the same semantics as local functions calls.

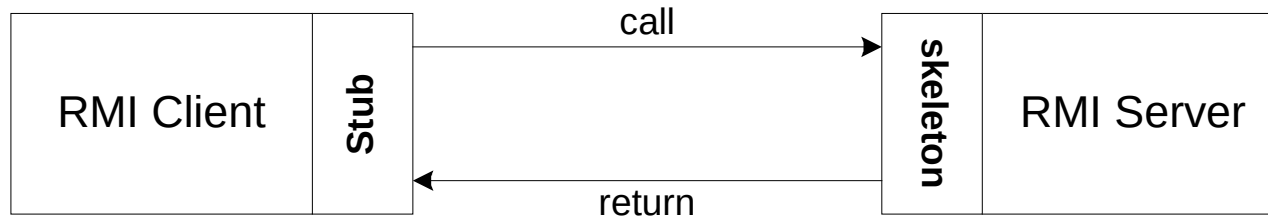


The General RMI Architecture

- The server must first bind its name to the registry
- The client lookup the server name in the registry to establish remote references.
- The Stub serializing the parameters to skeleton, the skeleton invoking the remote method and serializing the result back to the stub.



The Stub and Skeleton



- A client invokes a remote method, the call is first forwarded to stub.
- The stub is responsible for sending the remote call over to the server-side skeleton
- The stub opening a socket to the remote server, marshaling the object parameters and forwarding the data stream to the skeleton.
- A skeleton contains a method that receives the remote calls, unmarshals the parameters, and invokes the actual remote object implementation.

Steps for Developing an RMI System

1. Define the remote interface
2. Develop the remote object by implementing the remote interface.
3. Develop the client program.
4. Compile the Java source files.
5. Generate the client stubs and server skeletons.
6. Start the RMI registry.
7. Start the remote server objects.
8. Run the client

Step 1: Defining the Remote Interface

- To create an RMI application, the first step is the defining of a remote interface between the client and server objects.

```
/* SampleServer.java */  
import java.rmi.*;  
  
public interface SampleServer extends Remote  
{  
    public int sum(int a,int b) throws RemoteException;  
}
```

Step 2: Develop the remote object and its interface

- The server is a simple unicast remote server.
- Create server by extending `java.rmi.server.UnicastRemoteObject`.
- The server uses the `RMISecurityManager` to protect its resources while engaging in remote communication.

```
/* SampleServerImpl.java */
import java.rmi.*;
import java.rmi.server.*;
import java.rmi.registry.*;

public class SampleServerImpl extends UnicastRemoteObject
                                implements SampleServer
{
    SampleServerImpl() throws RemoteException
    {
        super();
    }
}
```

Step 2: Develop the remote object and its interface

- Implement the remote methods

```
/* SampleServerImpl.java */  
public int sum(int a,int b) throws RemoteException  
{  
    return a + b;  
}  
}
```

- The server must bind its name to the registry, the client will look up the server name.
- Use `java.rmi.Naming` class to bind the server name to registry. In this example the name call “SAMPLE-SERVER”.
- In the main method of your server object, the RMI security manager is created and installed.

Step 2: Develop the remote object and its interface

```
/* SampleServerImpl.java */
public static void main(String args[])
{
    try
    {
        System.setSecurityManager(new RMISecurityManager());
        //set the security manager

        //create a local instance of the object
        SampleServerImpl Server = new SampleServerImpl();

        //put the local instance in the registry
        Naming.rebind("SAMPLE-SERVER" , Server);

        System.out.println("Server waiting.....");
    }
    catch (java.net.MalformedURLException me)          {
        System.out.println("Malformed URL: " + me.toString());    }
    catch (RemoteException re) {
        System.out.println("Remote exception: " + re.toString()); }
}
```

Step 3: Develop the client program

- In order for the client object to invoke methods on the server, it must first look up the name of server in the registry. You use the `java.rmi.Naming` class to lookup the server name.
- The server name is specified as URL in the from (`rmi://host:port/name`)
- Default RMI port is 1099.
- The name specified in the URL must exactly match the name that the server has bound to the registry. In this example, the name is “SAMPLE-SERVER”
- The remote method invocation is programmed using the remote interface name (`remoteObject`) as prefix and the remote method name (`sum`) as suffix.

Step 3: Develop the client program

```
import java.rmi.*;
import java.rmi.server.*;
public class SampleClient
{
    public static void main(String[] args)
    {
        // set the security manager for the client
        System.setSecurityManager(new RMISecurityManager());
        //get the remote object from the registry
        try
        {
            System.out.println("Security Manager loaded");
            String url = "///localhost/SAMPLE-SERVER";
            SampleServer remoteObject = (SampleServer)Naming.lookup(url);
            System.out.println("Got remote object");
            System.out.println(" 1 + 2 = " + remoteObject.sum(1,2) );
        }
        catch (RemoteException exc) {
            System.out.println("Error in lookup: " + exc.toString()); }
        catch (java.net.MalformedURLException exc) {
            System.out.println("Malformed URL: " + exc.toString()); }
        catch (java.rmi.NotBoundException exc) {
            System.out.println("NotBound: " + exc.toString());
        }
    }
}
```

Web services

Web services

- Introduction
- Web services
- XMLRPC

Web Services

- Web e-commerce
- Based on open systems: XML, HTTP
- Interoperability
- Enterprise adopted standard: Microsoft, IBM, Sun, ...

SOAP: another standard?

- **Lack of standards harm interoperability**
- DCOM - DCE/NDR
- CORBA - IIOP/CDR
- JAVA - RMI/JRMP
- Netsolve – XDR

SOAP ?

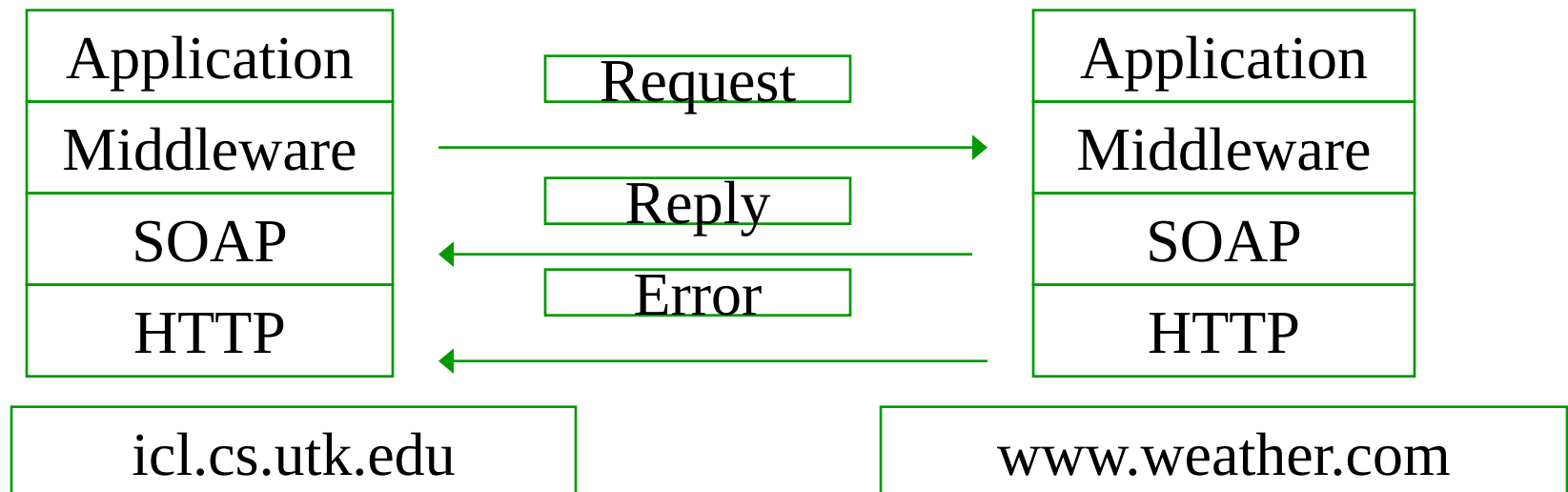
- **Security problems**
- Firewalls restrict traffic
- HTTP,FTP
- Most middleware systems assign port dinamically
- Real problem for internet applications

Potential solutions; REST

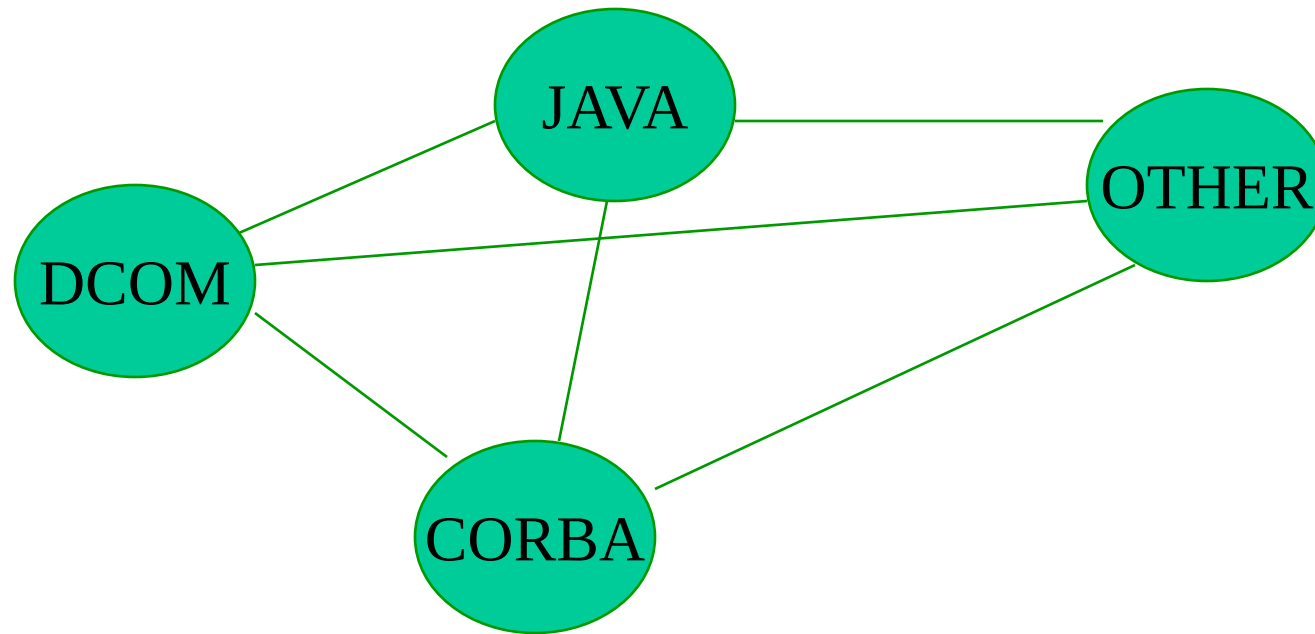
- ASCII
 - Raw text, lacks expresiveness
- HTTP, CGI resources
 - FTTP + ASCII [JSON]

Example

- `www.weather.com`
- `float CurrentTemp(zip_code)`



SOAP: Interoperability



Interfaces: WSDL

- Web Services Description Language
 - From WSDL to lenguaje XXX (stub cliente)
 - From Lenguaje YYY to interface WSDL
- We can locate and lookup web services with the WSDL URL
- Ej: www.xmethods.com (GEOIP)

UDDI

1.



Software companies, standards bodies, and developers populate the registry with descriptions (specifications) of types of services

2.



Implementers populate the registry with descriptions of their businesses and the services they expose

UDDI Registry

Implementations

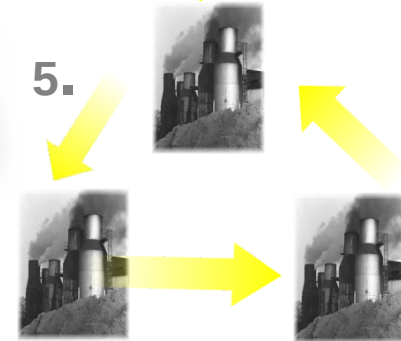
Service Types

3. UDDI Registry assigns a programmatically unique identifier to each service and registration

4.

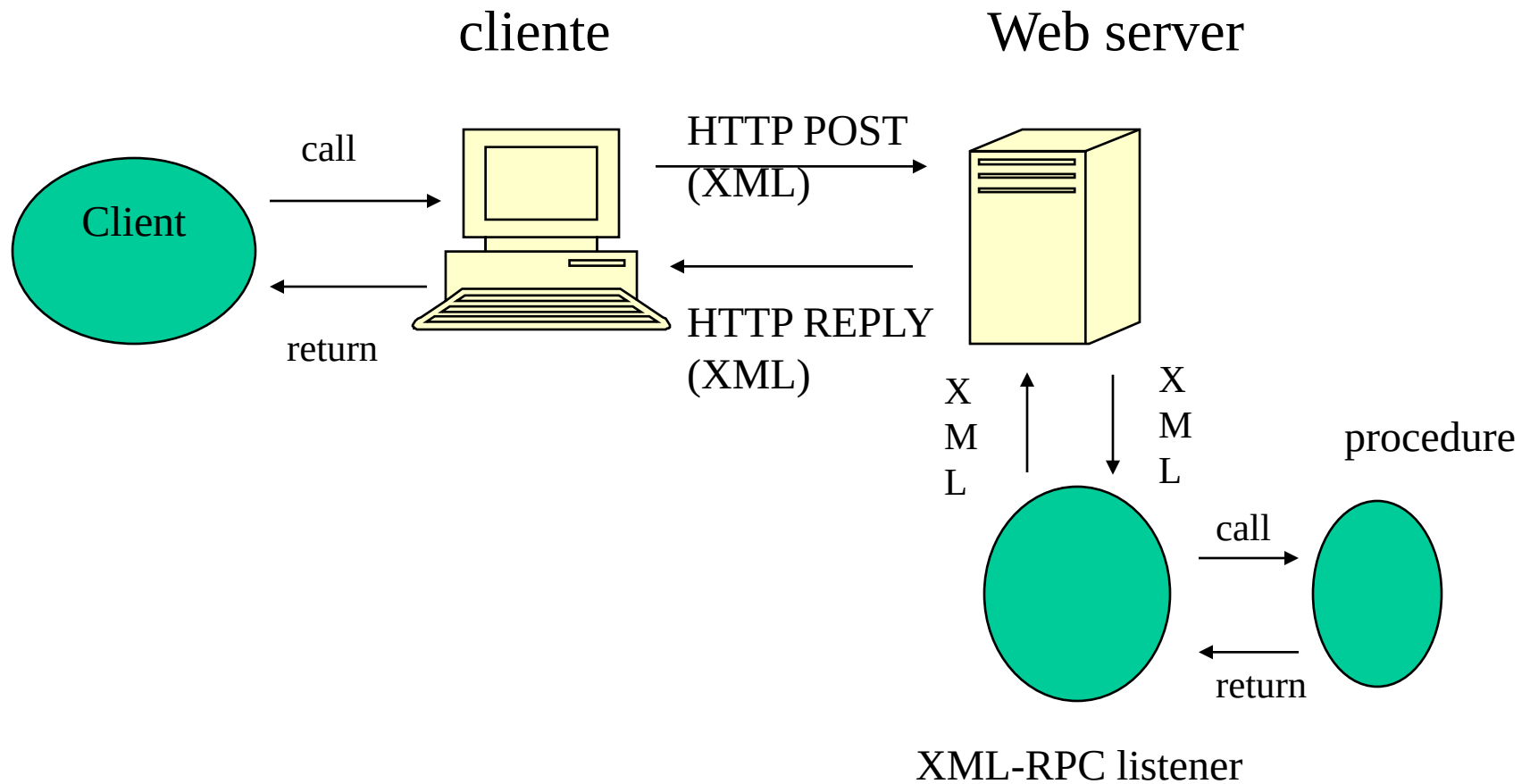
Marketplaces, search engines, and business applications query the registry to discover services at other companies and to facilitate integration

5.



Large businesses apply the same architecture and technologies internally

XML-RPC



Request

```
POST /RPC2 HTTP/1.0
User-Agent: Frontier/5.1.2
Host: xml.colorado.edu
Content-Type: text/xml
Content-length: 181
<?xml version="1.0"?>
<methodCall>
  <methodName>examples.getStateName</methodName>
  <params> <param>
    <value><i4>41</i4></value>
  </param></params>
</methodCall>
```

Scalar values <value>s

<i4> or <int>	four-byte signed integer	-12
<boolean>	0 (false) or 1 (true)	1
<string>	ASCII string	Hi!
<double>	double-precision	3.1415
<dateTime.iso8601>	date/time	19980717T14:08:55
<base64>	base64-encoded binary	eW91IGNhbid

Structured data- <struct>

<struct> contains <member>s, each <member> contains a <name> and a <value>

```
<struct>
  <member>
    <name>lowerBound</name>
    <value><i4>18</i4></value>
  </member>
  <member>
    <name>upperBound</name>
    <value><i4>122</i4></value>
  </member>
</struct>
```

Arrays- <array>

An <array> contains a single <data>, including a list of values <value>s

<array>

<data>

<value><i4>12</i4></value>

<value><string>Egypt</string></value>

<value><boolean>0</boolean></value>

<value><i4>-31</i4></value>

</data>

</array>

Reply sample

HTTP/1.1 200 OK

Connection: close

Content-Length: 158

Content-Type: text/xml

Date: Wed, 26 Sep 2001 10:10:28 GMT

Server: UserLand Frontier/5.1.2-WinNT

```
<?xml version="1.0"?>
```

```
<methodResponse>
```

```
  <params> <param>
```

```
    <value><string>South Dakota</string></value>
```

```
  </param></params>
```

```
</methodResponse>
```

Implementations

- 1 AOLserver: client/server (Aaron Swartz, Dave Bauer, Jerry Asher)
- 2 Apache: client/server (Andrew Bihun)
- 3 AppleScript: client (Late Night Software)
- 4 ASP: client/server (David Carter-Tod)
- 5 Axapta: client/server (Greg Pierce)
- 6 C/C++: client/server (Eric Kidd/FirstPeer)
- 7 C: client/server (Epinions)
- 8 C++: client (eZ systems)
- 9 Cold Fusion: client/server (Rob Blitz)
- 10 COM: client/server (Jan G.P. Sijm)
- 11 COM: client (Ignacio Vazquez)
- 12 Delphi/Kylix: client/server (Cliff Baeseman)
- 13 Dylan: client/server (Chris Double)
- 14 Eiffel: client/server (Glenn Maughan)
- 15 Flash: client (Patrick O'Lone)
- 16 Frontier/Radio UserLand: client/server (UserLand)
- 17 Guile: client/server (Evan Prodromou)
- 18 Internet Explorer: client (Olav Junker Kjær)
- 19 J2ME: client (Kyle Gabhart)
- 20 Java: client/server (Apache)
- 21 Java: client (Josh Lucas)
- 22 Java: client/server (Greger Ohlson)
- 23 Java: client/server (Hannes Wallnöfer)
- 24 Java: client/server (ThingMachines)
- 25 JavaScript: client (Scott Andrew)
- 26 JavaScript: client (Ruben Daniels)
- 27 K: client/server (Christian Langreiter)
- 28 KDE: client/server (Kurt Granroth)
- 29 Lingo: client (Stefan Rinner)
- 30 Lisp: client/server (Chris Double)
- 31 Lisp: client/server (Daniel Lundin)
- 32 Macintosh OS X: client/server (Apple Computer)
- 33 Microsoft .NET: client/server (Charles Cook)
- 34 Mozilla: client (Samuel Sieb)
- 35 Objective C: client/server (Marcus Müller)
- 36 Perl: client/server (Ken MacLeod)
- 37 Perl: client/server (Paul Kulchenko)
- 38 Perl: client/server (Randy Ray)
- 39 PHP: client/server (Epinions)
- 40 PHP: client/server (Useful Inc)
- 41 PHP: client/server. (Bård Farstad)
- 42 PHP: client/server (Stefan Saasen)
- 43 PHP: client/server (eZ systems)
- 44 Pike: server (Pitrich Information Technologies)
- 45 Python: server (Ken McIvor)
- 46 Python: client/server (PythonWare)
- 47 Python: client/server (Shilad Sen)
- 48 Real Basic: client (Sam DeVore)

Implementations

- Rebol: client/server (Chris Langreiter)
- Rebol: client/server (Thomas Jensen)
- Rebol: client/server (Andreas Bolka)
- Ruby: client/server (Michael Neumann)
- Scheme: client/server (Matt Jadud)
- Scheme: client (Pete Hopkins)
- Squeak: client (Christian Langreiter)
- Tcl: client (Vitessa)
- Tcl: client/server (Eric Yeh)
- Tcl: client/server (Steve Ball)
- Tintware: client/server (Michael K Montague)
- WebObjects: client/server (Hannes Wallnöfer/Jim Roepcke)
- Zope: client/server (PythonWare/Eric Kidd/UserLand)